SECURITY AUDIT

Symbiotic

Async/Account , Scope 2

FINAL REPORT

July 2026

BAILSEC.IO

Disclaimer:

BailSec was engaged by the client to perform a time-boxed technical security assessment of the target identified in this report. The assessment applies only to the scope, version, commit, deployment, materials and information identified in the report. Its findings are point-in-time and not exhaustive. Changes made after the assessment—including remediation changes—were not assessed unless expressly stated otherwise and may affect the applicability of the report's conclusions.

No security assessment can identify every vulnerability or guarantee that a system is secure, error-free or protected against attack or loss. Only the components, assumptions, threat models and risk categories expressly identified as in scope were assessed. The report should be read in full, including its scope, exclusions, unresolved findings and remediation status. The client remains responsible for design, testing, remediation, deployment, operation, monitoring and incident response.

This report is a technical assessment, not a certification, attestation, investment recommendation, or legal, financial, regulatory or tax advice. It does not constitute an endorsement of the client, the assessed system or any related project.

This report was prepared for the client under a separate engagement agreement. If published, its public availability is for transparency and informational purposes only. To the fullest extent permitted by applicable law, publication does not create an auditor-client relationship, duty of care or right of reliance in favor of any third party. Other readers should conduct their own diligence and obtain appropriate professional advice.

Only the complete, unmodified report as issued by BailSec is authoritative. The signed engagement agreement exclusively governs the contractual rights and obligations between BailSec and the client, including warranties, remedies, liability limitations, indemnities and dispute resolution. Nothing in this notice excludes liability that cannot lawfully be excluded.

1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Symbiotic – Async/Account – Audit Report – Scope 2
Website	symbiotic.fi
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/symbioticfi/core-mirror/blob/5752d056ba8de336d6f0b9716aef4a7b78b8a73b/src/contracts/adapters/ThreeFAdapter.sol
Resolution 1	https://github.com/symbioticfi/core-mirror/blob/d18d4699c6a48f1efad8d7904a6e3427cd0dfd5a/src/contracts/adapters/ThreeFAdapter.sol

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High						
Medium	1			1		
Low	4			4		
Informational	7	1		6		
Governance						
Total	12	1		11		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

ThreeFAdapter

The ThreeFAdapter contract is a VaultV2 adapter that deploys vault assets into whitelisted 3F bridge facilitator requests. It signs offers as an ERC-1271 signer through isValidSignature using the configurable offerSigner. onRequestConsumed is the whitelisted request callback that enforces per-request bounds and minimum yield, then pulls principal from the vault via allocateExact inside a transient window and approves the request. finalizeRequest permissionlessly burns the adapter's PT and YT shares in a matured request back into assets and drops the request from tracking. totalAssets values live requests, getMaxAssets reports remaining deployable capacity, and the owner sets the signer and limits. The live principal is illiquid until finalization, so _deallocate returns zero.

Appendix: Just-in-time allocation

Allocation is driven by the request rather than by the delegator. During onRequestConsumed the adapter sets the transient flag _inConsume, which switches allocatable from zero to the base implementation, calls allocateExact on the delegator to pull exactly the principal from the vault, then clears the flag and approves the calling request for the principal. Outside this window, the adapter cannot receive vault allocations at all.

Privileged Functions

- setOfferSigner
- setLimitsPerRequest

Invariants

- allocatable is zero except inside the _inConsume window opened during onRequestConsumed.
- Each live request appears exactly once in the requests array, and requestIndex maps it to its array position plus one; a zero index means the address is not a tracked request.
- requests.length never exceeds MAX_REQUESTS.
- Every accepted request satisfies minAssetsPerRequest <= principal <= maxAssetsPerRequest, and its yield scaled by YIELD_PRECISION over principal is at least minYieldPerRequest.
- pendingAssets for a request is set when the request is consumed and cleared to zero when it is finalized or when the request address is not tracked.
- Principal deployed into a live request cannot be deallocated back to the vault; deallocate only ever returns assets that are free in the adapter.

Issue_01	totalAssets jumps when canWithdraw flips
Severity	**Medium**
Description	When a request is funded, the contract stores its value in pendingAssets, equal to the amount funded. It values the YT tokens at 0. The same is done in the request's convertToAssets function. When the Request's loan is repaid, the convertToAssets function now evaluates the PT and YT tokens at the actual repayment rate. This causes a sudden change in the totalAssets value the moment the canWithdraw flag in the request contract is flipped. Thus, all the yield in the Adapter is accrued in a single transaction. This makes the vault susceptible to sandwich attacks. Users can frontrun the canWithdraw flip to deposit funds into the vault at a lower asset-per-share and then withdraw immediately after at a higher asset-per-share, and capture most of the yield generated.
Recommendations	Consider implementing a way to drip the yield into the contract gradually after finalization, instead of realizing the profit in a single transaction.
Comments / Resolution	Acknowledged.

Issue_02	Matured 3F request value is counted in NAV, but is unavailable for withdrawals until finalization
Severity	Low
Description	Once a 3F request becomes withdrawable, ThreeFAdapter.totalAssets values the request through the adapter's PT/YT balances and the request's convertToAssets result. At that point, the matured request value is included in vault NAV. However, the value is still not available as vault liquidity. ThreeFAdapter._deallocate always returns 0, so the delegator cannot pull matured request value back to the vault through normal deallocation. The assets only become adapter-held free assets after someone calls finalizeRequest, which burns the adapter's PT/YT position in the request. This creates a mismatch between accounting and liquidity. The vault can include matured 3F value in totalAssets, while withdrawable cannot use that value to satisfy withdrawals.
Recommendations	Consider acknowledging and documenting this behaviour, or consider adding the finalizeRequest to the deallocation flow.
Comments / Resolution	Acknowledged.

Issue_03	Loan defaults can be bypassed by frontrunning finalizeWithdraw calls
Severity	**Low**
Description	When finalizeRequest is called, the contract redeems the PT and YT tokens into the base asset. Normally, this is profitable, but in case of defaults, this can evaluate to less than pendingAssets amount of assets. Since this loss is realized in a single transaction, users can bypass this loss by frontrunning this call and withdrawing assets before this call, forcing the other holders to bear this loss instead.
Recommendations	Consider adding a delay in vault withdrawals or acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_04	getMaxAssets ignores per-request principal limits, and request length limit
Severity	Low
Description	getMaxAssets is meant to tell 3F requestors the largest principal amount that can be funded into a new request right now. It only considers delegator headroom and vault withdrawable liquidity. It does not apply maxAssetsPerRequest or minAssetsPerRequest, even though onRequestConsumed enforces both on every consume. That mismatch breaks the function's stated contract. A requestor that sizes an offer from getMaxAssets can still revert on consume for reasons the view never signals. If capacity is 50 but minAssetsPerRequest is 100, getMaxAssets returns 50, even though a request of that size is not possible. Furthermore, if the requests.length matches MAX_REQUESTS already; no further requests can be consumed, irrespective of what getMaxAssets returns.
Recommendations	Update getMaxAssets to apply the same per-request bounds used in onRequestConsumed. Cap the result at maxAssetsPerRequest. If the capped value is below minAssetsPerRequest, or if requests is already at the max limit, return 0 so callers know no valid request size is available.
Comments / Resolution	Acknowledged, documented.

Issue_05	Request finalization accepts zero or partial recovery without checks or loss reporting
Severity	Informational
Description	finalizeRequest calls burnAll on the 3F request but ignores the returned principal and yield amounts. It always clears pendingAssets, removes the request from tracking, and emits FinalizeRequest with no recovered amount. The adapter never checks whether recovery is below the original principal, whether nothing was returned at all, or whether finalization left the adapter with the expected assets. If a 3F request defaults or repays less than expected, finalization still succeeds. The vault keeps no on-chain record of how much was lost.
Recommendations	Consider documenting and acknowledging this issue, or reverting and requiring explicit handling in case of a shortfall during finalization.
Comments / Resolution	Acknowledged.

Issue_06	Zero-principal requests can consume adapter request slots
Severity	**Informational**
Description	onRequestConsumed only rejected requests where principalAssets is below minAssetsPerRequest. If minAssetsPerRequest is set to zero, a whitelisted request can consume with principalAssets = 0. The yield check is skipped when the principleAssets is zero, but the request is still added to requests and consumes one of the MAX_REQUESTS slots This can let zero-principal requests fill adapter capacity until they are finalized.
Recommendations	Rejects zero-principal requests explicitly, or require minAssetsPerRequest to be greater than zero.
Comments / Resolution	Acknowledged.

Issue_07	getMaxAssets triggers permissionless sweepPending side effects
Severity	**Informational**
Description	ThreeFAdapter.getMaxAssets looks like a capacity query but is not read-only. Every call runs UniversalDelegator.sweepPending, which processes the vault's pending withdrawal queue: sweeping free assets from adapters, filling the queue, and requesting deallocation from others as needed. Because sweepPending is permissionless, any account can trigger this sweep by calling getMaxAssets. Integrators using staticcall will revert on a state-changing call. Those using a normal call mutate vault state while only trying to read capacity.
Recommendations	Document that getMaxAssets is state-mutating and must not be called via staticcall.
Comments / Resolution	Acknowledged, documented.

Issue_08	Offer signer rotation invalidates pending offers with no grace period
Severity	**Informational**
Description	The owner can change offerSigner at any time via setOfferSigner, with no delay or overlap window. Offers must be signed by the current signer, and the 3F Request contract checks that inside consume before calling onRequestConsumed. That check and callback run in the same transaction, so offerSigner cannot change between them on-chain. The issue is rotation timing: offers signed under the old key but not yet consumed will fail once the signer is updated. When consume runs, validation uses the new signer. Old signatures revert with an invalid signature error before onRequestConsumed runs. No funds move, but the offer cannot be completed.
Recommendations	Consider timelocking setOfferSigner and clearing pending offers before rotating. Document that rotation immediately invalidates unconsumed offers signed under the previous key.
Comments / Resolution	Acknowledged, documented.

Issue_09	Stale offers fail when limits change without a transition period
Severity	**Informational**
Description	Off-chain 3F tooling sizes signed offers using getMaxAssets and the adapter's per-request limits. Both read live state: delegator headroom, vault withdrawable balance, sweep status, and current min/max/yield settings. All of these can change instantly, invalidating any in-flight signed offers instantly. The owner can call setLimitsPerRequest, the delegator admin can call setLimits, and capacity can shrink when other requests fill, liquidity drops, or a sweep is pending. There is no timelock or pending-limit state. If the pre-signed offers are used, it will lead to a revert.
Recommendations	Consider adding a timelock for changing the parameters, or managing the coordination of this off-chain.
Comments / Resolution	Acknowledged.

Issue_10	Adapter rejects compact ERC-2098 EOA signatures accepted by 3F request contracts
Severity	Informational
Description	ThreeFAdapter.isValidSignature validates offers by delegating to OpenZeppelin SignatureChecker.isValidSignatureNow(offerSigner, hash, signature). In this checkout, the OpenZeppelin EOA path calls ECDSA.tryRecover(hash, bytes signature), which only accepts 65-byte ECDSA signatures. The 3F request contracts use Solady's SignatureCheckerLib, whose EOA path accepts both standard 65-byte signatures and 64-byte ERC-2098 compact signatures. When the adapter is the offer.maker, the request validates the offer through the adapter's ERC-1271 isValidSignature function. As a result, a compact 64-byte signature can be rejected by the adapter.
Recommendations	Consider supporting ERC-2098 compact signatures in ThreeFAdapter.isValidSignature, by using the same checker as that of 3F. Alternatively, document that the 3F adapter only supports 65-byte EOA signatures.
Comments / Resolution	Acknowledged, documented.

Issue_11	Adapter cannot selectively cancel its own 3F offer nonces
Severity	**Informational**
Description	3F request contracts support offer cancellation through setNonce(newNonce). The nonce is keyed by msg.sender, meaning only the maker address can advance its own nonce. For offers where ThreeFAdapter is the maker, the nonce owner is the adapter contract itself. The adapter exposes setOfferSigner, but it does not expose any function that forwards a nonce update to a 3F request contract. This prevents selective cancellation of a specific pending offer nonce for a specific request through the adapter. Operators must rely on broader controls such as rotating offerSigner, setting the signer to zero, changing request limits, or waiting for offer expiration.
Recommendations	Add an owner-only adapter function that calls setNonce(newNonce) on a whitelisted 3F request. Validate that the target request is whitelisted and uses the vault asset. Document signer rotation as a coarse emergency cancellation path, not a replacement for request-specific nonce management.
Comments / Resolution	Fixed following recommendations.

Issue_12	setRequestNonce cannot cancel offers while request is paused or not whitelisted
Severity	Low
Description	setRequestNonce reuses _validateRequest which only accepts requests with active whitelisted status and matching asset. If a request is paused or removed from the whitelist, the adapter owner cannot advance the nonce for that request until it is enabled/whitelisted.
Recommendations	Consider allowing nonce cancellation for requests that are not enabled or whitelisted.
Comments / Resolution	Acknowledged.